

CS6.302 - Software System Development

Assignment 1 - Database Design

Total Marks: 100

Duration: 2 Weeks

Due Date: 4 September 2026

About the Assignment

- This is a Team Assignment with team size of 4
- Please follow submission guidelines while submitting the assignment.
- Submissions are accepted via Moodle and your shared github repository.
- All questions are self explanatory. All assumptions are open to your interpretation. Hence, ensure that you list down your assumptions while responding to the questions.

Project to be chosen

There are 5 projects given in this assignment. Each team is assigned ONLY one project which they need to do.

The project number for each team is: **project_no** = (**team_no** % 5) + 1.

Eg: Team 14 will do Project number 5.

Submission Guidelines

To satisfy the submission requirements and grading criteria, your GitHub repository structure should look like this:

```
<roll_number_al>
├── README.md                                # Setup steps, assumptions, & EXPLAIN plans
├── docs/
│   ├── relational_erd.png                  # Visual diagram of PostgreSQL schema
│   └── mongo_schema_map.json              # Document structure & validation models
├── sql/
│   ├── 01_schema_ddl.sql                  # Tables, data types, PK/FKs, CHECK constraints
│   ├── 02_indexes.sql                    # Partial indexes and secondary indexes
│   ├── 03_triggers_and_audit.sql          # Audit log triggers
│   ├── 04_stored_procedures.sql          # Workflow 1: Atomic transaction procedure
│   ├── 05_materialized_views.sql         # Materialized view definitions & refresh
│   └── 06_window_analytics.sql           # Workflow 2: Window functions, CTEs
├── mongo/
│   ├── 01_collections_and_indexes.js      # 2dsphere and TTL index creation
│   ├── 02_workflow3_geonear.js           # Workflow 3: Geospatial aggregation ($geoNear)
│   └── 03_workflow4_facet.js              # Workflow 4: Multi-faceted analytics ($facet)
├── data_generation/
│   ├── postgres_seeder.py                # Generates 100k+ ledger/audit rows
│   ├── mongo_seeder.py                   # Generates 500k+ geospatial telemetry pings
│   └── requirements.txt                    # Python dependencies
└── performance/
```

└─ postgres_explain_analyzes.txt	# Raw EXPLAIN (ANALYZE) logs
└─ mongo_execution_stats.json	# Raw explain("executionStats") JSON

Your team must submit a GitHub repository containing:

1. **ERDs & Schema Maps:** Visual maps of your Postgres relational schema and Mongo document structures.
2. **Setup Scripts:** Full DDL (CREATE TABLE, INDEX, etc.) and Mongo collection/index creation commands.
3. **Workflow Scripts:** All stored procedures, triggers, materialized views, and MongoDB aggregation pipelines saved as independent .sql and .js files.
4. **Data Generation Code:** The scripts used to seed your 100k+ rows of mock data.
5. **Performance Proof (CRITICAL):** In your README.md, you must paste the output of EXPLAIN ANALYZE (Postgres) and explain('executionStats') (Mongo) for your heaviest queries (Workflows 2, 3, and 4) proving they hit the indexes and avoid sequential/collection scans.

In Moodle:

1. Submit a single ZIP file named <team_number_a1>.zip (must be strictly under **20 MB**).
2. **DO NOT** include raw database dump files (.sql data dumps, .csv, or exported Mongo collections). Only submit the data generation scripts (postgres_seeder.py, mongo_seeder.py).
3. **DO NOT** include Python virtual environments (venv/) or cache directories (__pycache__/).
4. Ensure your README.md includes the direct URL to your team's GitHub repository along with the specific final commit hash.

Grading Rubric (100 Points Total)

- **(20 points) Database Engine Logic:** Proper implementation of stored procedures, TRIGGER audit logging, and CHECK constraints.
- **(25 points) Advanced Analytics:** Flawless execution of SQL Window Functions/CTEs and MongoDB multi-faceted (\$facet) aggregation pipelines.
- **(10 points) Indexing & Query Optimization:** Proper creation and verification of partial indexes, materialized views, and TTL/Geospatial indexes.
- **(10 points) Stress Testing & Proof:** Generation of 100k+ mock rows and valid EXPLAIN execution plans documented in the README.
- **(35 points) Viva Voce:** 15 points for technical questioning and 20 points for live demonstration of the executed scripts and working database technology.

Objective

Design and deploy a production-grade database managing persistence across **PostgreSQL** and **MongoDB**. Since there is no front end application code for this assignment, you will tackle real-world data engineering challenges entirely at the database level. This includes transactional stored procedures, conditional partial indexing, multi-stage MongoDB aggregations with \$facet, materialized views, and geospatial indexing (2dsphere) for logistics.

Instructions: Your team must select **ONE** of the 5 projects listed below. Implement the specific schema, database tasks, and workflows for your chosen project.

PROJECT 1: BiteStream – Food Delivery & Real-Time Logistics

Step 1: Database Provisioning & Schema Constraints

1. Structured Database (PostgreSQL):

- **users:** id (UUID/INT, PK), name (VARCHAR), wallet_balance (DECIMAL(10,2) CHECK (wallet_balance ≥ 0.00))
- **wallet_audit_logs:** id (UUID/INT, PK), user_id (FK to users), amount_changed (DECIMAL(10,2)), action_type (DEBIT, CREDIT), balance_after, timestamp (TIMESTAMPTZ DEFAULT NOW())
- **restaurants:** id (UUID/INT, PK), name (VARCHAR), latitude (FLOAT), longitude (FLOAT)
- **orders:** id (UUID/INT, PK), user_id (FK), restaurant_id (FK), total_amount (DECIMAL(10,2)), status (VARCHAR: PREPARING, DELIVERING, DELIVERED), created_at (TIMESTAMPTZ)

2. Unstructured Database (MongoDB):

- **Menus:** Flexible catalog documents with nested items, categories, and customization additions.
- **Reviews:** Structured reviews with ratings, sentiment tags, and timestamps.
- **DriverPings:** Real-time geospatial location logs with GeoJSON Point format.

Step 2: Database-Heavy Engineering Tasks

- **PostgreSQL Trigger (Audit Logging):** Implement an automated PostgreSQL TRIGGER on the users table that fires AFTER UPDATE OF wallet_balance. It must automatically record an immutable record into wallet_audit_logs.
- **Partial Indexing:** Create a PostgreSQL partial unique index prohibiting a user from having more than one active delivery at a time:

```
CREATE UNIQUE INDEX idx_active_user_order ON orders (user_id) WHERE status IN ('PREPARING', 'DELIVERING');
```
- **Materialized View:** Create a PostgreSQL MATERIALIZED VIEW joining restaurants with their total completed order count and revenue. Write a function to REFRESH CONCURRENTLY this view.
- **MongoDB Geospatial & TTL Indexes:** Create a 2dsphere index on DriverPings.location. Create a TTL index on DriverPings.created_at with expireAfterSeconds: 7200 (2 hours).

Step 3: Complex Database Workflows (Pure Scripts)

- **Workflow 1: Atomic Checkout (Stored Procedure):** Write a PL/pgSQL Stored Procedure sp_execute_checkout. It must start a REPEATABLE READ transaction, deduct the wallet balance (verifying CHECK constraints), insert an order into the orders table (which triggers the audit log), and COMMIT. If the balance is insufficient, catch the exception and ROLLBACK.
- **Workflow 2: SQL Window Analytics:** Write a .sql script utilizing CTEs and Window Functions: Calculate the 7-day moving average of order revenue per restaurant, and use DENSE_RANK() to rank top venues.

- **Workflow 3: Nearest Active Driver:** Write a MongoDB .js pipeline using \$geoNear to locate the closest active driver within a 5km radius of a restaurant's coordinates.
 - **Workflow 4: Multi-Faceted Review Analytics:** Write a MongoDB .js pipeline using \$facet to compute: 1) Rating distribution (1-5 stars), 2) Most frequent tag strings via \$unwind, 3) Overall average rating.
-

PROJECT 2: RideSync – Global Ride-Hailing Network

Step 1: Database Provisioning & Schema Constraints

1. Structured Database (PostgreSQL):

- **riders:** id (UUID/INT, PK), name (VARCHAR), wallet_balance (DECIMAL(10,2) CHECK (wallet_balance ≥ 0.00))
- **wallet_audit_logs:** id (UUID/INT, PK), rider_id (FK), amount_changed, action_type, balance_after, timestamp
- **vehicles:** id (UUID/INT, PK), license_plate (VARCHAR), class (VARCHAR), is_active (BOOLEAN)
- **trips:** id (UUID/INT, PK), rider_id (FK), vehicle_id (FK), fare_amount, status (REQUESTED, IN_TRANSIT, COMPLETED), created_at

2. Unstructured Database (MongoDB):

- **VehicleMetadata:** Flexible documents storing inspection records and dynamic features.
- **TripReviews:** Structured reviews with ratings, driver feedback tags, and timestamps.
- **TelemetryPings:** Real-time geospatial location logs with GeoJSON Point format.

Step 2: Database-Heavy Engineering Tasks

- **PostgreSQL Trigger (Audit Logging):** Implement a TRIGGER on riders firing AFTER UPDATE OF wallet_balance to insert immutable logs into wallet_audit_logs.
- **Partial Indexing:** Create a PostgreSQL partial unique index prohibiting a rider from being in multiple active trips:

```
CREATE UNIQUE INDEX idx_active_rider_trip ON trips (rider_id)
WHERE status IN ('REQUESTED', 'IN_TRANSIT');
```
- **Materialized View:** Create a MATERIALIZED VIEW joining vehicles with their lifetime trip count and total earnings. Write a REFRESH CONCURRENTLY function.
- **MongoDB Geospatial & TTL Indexes:** Create a 2dsphere index on TelemetryPings.location and a 2-hour TTL index on created_at.

Step 3: Complex Database Workflows (Pure Scripts)

- **Workflow 1: Atomic Booking (Stored Procedure):** Write a PL/pgSQL Stored Procedure that starts a transaction, places the rider's `wallet_balance` into escrow, inserts a trip, and commits. Rolls back on CHECK constraint failure.
 - **Workflow 2: SQL Window Analytics:** Write a SQL script using CTEs and Window Functions to calculate the 7-day moving average of fare revenue per vehicle, ranked via `DENSE_RANK()`.
 - **Workflow 3: Nearest Available Vehicle:** Write a MongoDB pipeline using `$geoNear` to locate the closest available vehicle within 5km of a rider.
 - **Workflow 4: Multi-Faceted Review Analytics:** Write a MongoDB pipeline using `$facet` to compute rating distributions, most frequent feedback tags via `$unwind`, and overall average rating.
-

PROJECT 3: StaySpot – Vacation Rental & Experiences

Step 1: Database Provisioning & Schema Constraints

1. Structured Database (PostgreSQL):

- `guests`: `id` (UUID/INT, PK), `name` (VARCHAR), `wallet_balance` (DECIMAL(10,2) CHECK (`wallet_balance` $\geq$ 0.00))
- `wallet_audit_logs`: `id` (UUID/INT, PK), `guest_id` (FK), `amount_changed`, `action_type`, `balance_after`, `timestamp`
- `properties`: `id` (UUID/INT, PK), `title` (VARCHAR), `base_price`, `latitude`, `longitude`
- `bookings`: `id` (UUID/INT, PK), `guest_id` (FK), `property_id` (FK), `total_cost`, `status` (CONFIRMED, CHECKED_IN, COMPLETED), `created_at`

2. Unstructured Database (MongoDB):

- `PropertyAmenities`: Flexible catalog documents with nested arrays for house rules and accessibility features.
- `PropertyReviews`: Structured reviews with ratings, location tags, and timestamps.
- `SearchSessions`: Geospatial logs of where users are actively dropping pins to search for homes.

Step 2: Database-Heavy Engineering Tasks

- **PostgreSQL Trigger (Audit Logging):** Implement a TRIGGER on `guests` firing AFTER UPDATE OF `wallet_balance` to insert into `wallet_audit_logs`.
- **Partial Indexing:** Create a partial unique index prohibiting a guest from having multiple overlapping check-ins:

```
CREATE UNIQUE INDEX idx_active_stay ON bookings (guest_id)
WHERE status = 'CHECKED_IN';
```

- **Materialized View:** Create a `MATERIALIZED VIEW` summarizing properties by total nights booked and gross revenue. Write a `REFRESH CONCURRENTLY` function.
- **MongoDB Geospatial & TTL Indexes:** Create a `2dsphere` index on `SearchSessions.location` and a 2-hour TTL index on `created_at`.

Step 3: Complex Database Workflows (Pure Scripts)

- **Workflow 1: Atomic Booking (Stored Procedure):** Write a PL/pgSQL Stored Procedure executing a transaction to deduct the guest's balance, insert the booking, and commit. Must gracefully rollback on failure.
- **Workflow 2: SQL Window Analytics:** Use CTEs and Window Functions to calculate a 7-day moving average of booking revenue per property, ranked by `DENSE_RANK()`.
- **Workflow 3: Trending Search Hotspots:** Write a MongoDB `$geoNear` pipeline to cluster recent `SearchSessions` within a 5km radius of specific coordinates.
- **Workflow 4: Multi-Faceted Review Analytics:** Use `$facet` to output rating distributions, most frequent review tags (using `$unwind`), and overall average property rating.

PROJECT 4: CareConnect – On-Demand Telemedicine

Step 1: Database Provisioning & Schema Constraints

1. Structured Database (PostgreSQL):

- `patients`: `id` (UUID/INT, PK), `name` (VARCHAR), `hsa_balance` (DECIMAL(10,2) CHECK (`hsa_balance` $\geq$ 0.00))
- `wallet_audit_logs`: `id` (UUID/INT, PK), `patient_id` (FK), `amount_changed`, `action_type`, `balance_after`, `timestamp`
- `clinics`: `id` (UUID/INT, PK), `name` (VARCHAR), `latitude`, `longitude`, `is_accepting_patients` (BOOLEAN)
- `appointments`: `id` (UUID/INT, PK), `patient_id` (FK), `clinic_id` (FK), `copay_amount`, `status` (WAITING, IN_CONSULTATION, DISCHARGED), `created_at`

2. Unstructured Database (MongoDB):

- `MedicalCatalogs`: Flexible documents storing specialist availability and varied medication details.
- `PatientReviews`: Structured reviews with ratings, bedside-manner tags, and timestamps.
- `NursePings`: Real-time geospatial location logs of dispatched mobile nurses.

Step 2: Database-Heavy Engineering Tasks

- **PostgreSQL Trigger (Audit Logging):** Implement a `TRIGGER` on `patients` to log any changes to `hsa_balance` into `wallet_audit_logs`.

- **Partial Indexing:** Create a partial unique index preventing a patient from being in multiple concurrent consultations:

```
CREATE UNIQUE INDEX idx_active_consult ON appointments (patient_id) WHERE status IN ('WAITING', 'IN_CONSULTATION');
```
- **Materialized View:** Create a `MATERIALIZED VIEW` aggregating clinics by total monthly patient discharges. Write a `REFRESH CONCURRENTLY` function.
- **MongoDB Geospatial & TTL Indexes:** Create a `2dsphere` index on `NursePings.location` and a 2-hour TTL index on `created_at`.

Step 3: Complex Database Workflows (Pure Scripts)

- **Workflow 1: Atomic Appointment (Stored Procedure):** Write a PL/pgSQL Stored Procedure to safely deduct HSA balances, create the appointment, and log the audit trail atomically.
- **Workflow 2: SQL Window Analytics:** Write a query with CTEs and Window Functions to compute a 7-day moving average of copay revenue per clinic, ranked via `DENSE_RANK()`.
- **Workflow 3: Nearest Mobile Nurse:** Write a `$geoNear` MongoDB pipeline locating the closest active mobile nurse to a patient's coordinates.
- **Workflow 4: Multi-Faceted Review Analytics:** Use `$facet` in MongoDB to extract rating buckets, frequent sentiment tags via `$unwind`, and global average ratings.

PROJECT 5: GigTask – Freelance Micro-Jobs Marketplace

Step 1: Database Provisioning & Schema Constraints

1. Structured Database (PostgreSQL):

- `clients`: `id` (UUID/INT, PK), `name` (VARCHAR), `escrow_balance` (DECIMAL(10,2) CHECK (escrow_balance ≥ 0.00))
- `wallet_audit_logs`: `id` (UUID/INT, PK), `client_id` (FK), `amount_changed`, `action_type`, `balance_after`, `timestamp`
- `freelancers`: `id` (UUID/INT, PK), `name` (VARCHAR), `latitude`, `longitude`, `is_available` (BOOLEAN)
- `contracts`: `id` (UUID/INT, PK), `client_id` (FK), `freelancer_id` (FK), `budget`, `status` (FUNDED, IN_PROGRESS, COMPLETED), `created_at`

2. Unstructured Database (MongoDB):

- `Portfolios`: Flexible documents storing unstructured freelancer skills and certifications.
- `GigReviews`: Structured reviews with ratings, skill-tags, and timestamps.
- `WorkerLocations`: Real-time geospatial logs of active physical laborers in GeoJSON format.

Step 2: Database-Heavy Engineering Tasks

- **PostgreSQL Trigger (Audit Logging):** Implement a TRIGGER on clients logging escrow_balance updates to wallet_audit_logs.
- **Partial Indexing:** Create a partial unique index preventing a freelancer from working multiple overlapping active contracts:

```
CREATE UNIQUE INDEX idx_active_gig ON contracts (freelancer_id) WHERE status = 'IN_PROGRESS';
```
- **Materialized View:** Create a MATERIALIZED VIEW joining freelancers with their lifetime completed contracts and total earnings. Write a REFRESH CONCURRENTLY function.
- **MongoDB Geospatial & TTL Indexes:** Create a 2dsphere index on WorkerLocations.location and a 2-hour TTL index on created_at.

Step 3: Complex Database Workflows (Pure Scripts)

- **Workflow 1: Atomic Gig Funding (Stored Procedure):** Write a PL/pgSQL Stored Procedure that safely deducts client funds into escrow, creates a contract, and commits atomically.
 - **Workflow 2: SQL Window Analytics:** Utilize CTEs and Window Functions to compute the 7-day moving average of contract revenue per freelancer, ranked by DENSE_RANK().
 - **Workflow 3: Nearest Available Worker:** Write a \$geoNear pipeline to find the closest available freelancer to a physical job site.
 - **Workflow 4: Multi-Faceted Review Analytics:** Write a \$facet pipeline extracting rating distributions, top skill-tags via \$unwind, and overall worker ratings.
-

Step 4: Data Generation, Stress Testing & Submission

A database is only "heavy" if it contains significant data. You must generate mock data for your system to prove your queries and indexes work at scale.

- Generate at least **100,000 ledger/audit entries** and **50,000 orders/bookings** (PostgreSQL).
- Generate at least **500,000 geospatial pings** (MongoDB).
- *Hint:* Use Python scripts (Faker), mockaroo.com, or SQL generate_series() to populate your tables.